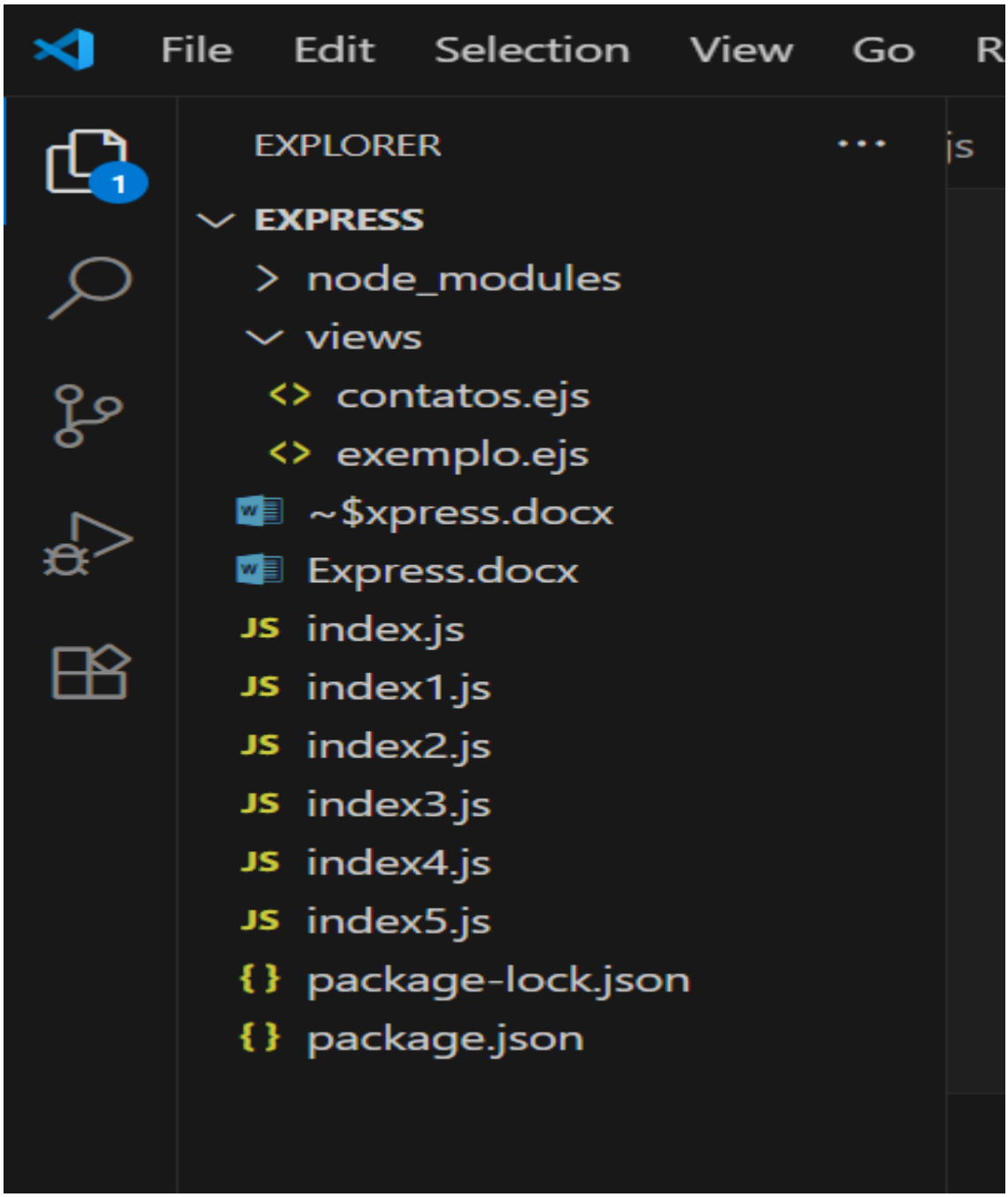
Aula 01

Framework Express.js



O que é um "framework"?

Um *framework* é um conjunto de ferramentas e bibliotecas que fornece uma estrutura base para o desenvolvimento de software. Ele facilita tarefas comuns, como:

- Criar rotas (URLs que respondem a requisições)
- Tratar requisições e respostas HTTP

- Trabalhar com middlewares (funções intermediárias para processar dados, autenticação, logs etc.)
- Lidar com formulários, cookies, erros etc.

O que o Express faz?

O **Express.js** permite que você crie servidores web e APIs REST de maneira simples e rápida. É muito usado em conjunto com outras tecnologias como MongoDB, React, Angular ou Vue (ex: no stack MERN).

Exemplo básico com Express:

Os servidores minimalistas são uma estratégia de desenvolvimento cada vez mais comum, estando presente em plataformas como: Python, com o Flask; Java, em que temos o Spring Boot; e NodeJS, que oferece o framework Express. São produtos similares, oferecendo o tratamento para requisições HTTP para todos os métodos do protocolo, além de permitirem, em maior ou menor escala, o uso de funcionalidades mais avançadas, como codificadores e autenticação OAuth. Para um sistema web tradicional, baseado em páginas, o requisito mínimo é o tratamento de rotas via métodos GET e POST, algo que o Express oferece, de forma simples, robusta e flexível, através de seu modelo de roteamento. Para facilitar a criação desse tipo de sistema, o uso de templates baseados em HTML é um grande facilitador, além de separar a lógica e o design de telas, permitindo especializar o desenvolvimento por perfil de profissional. Com os conhecimentos adquiridos, fomos capazes de criar um aplicativo web simples, mas que incorpora as técnicas necessárias para a definição de sistemas maiores, nos quais teríamos diversas páginas, mas continuaríamos a utilizar apenas os métodos GET e POST do protocolo HTTP.

Características Gerais do Express

Para que possamos responder às solicitações HTTP, precisamos de um servidor adequado, no qual eram utilizados quase que exclusivamente o Apache e o Internet Information Service (IIS), os principais Web Servers do mercado. Também temos diversos Application Servers, como JBoss, Glassfish, Payara e Web Sphere, os quais oferecem, além das respostas via HTTP, um ambiente de objetos distribuídos para aplicações corporativas.

Servidor Express

Com a evolução das tecnologias e a popularização dos microsserviços, é cada vez mais comum o uso de um servidor embarcado, no qual não é necessária uma estrutura externa, e o aplicativo já contém um servidor HTTP, com o mínimo de funcionalidades. Entre os exemplos de tecnologias, nesse perfil, podemos citar Spring Boot (Java), Flask (Python) e, no ambiente do NodeJS, Express. Express é um servidor minimalista, ou seja, ele utiliza o mínimo de recursos para executar, viabilizando sua inclusão no sistema que será desenvolvido. Além de não exigir um ambiente de execução externo, pode ser disponibilizado via Docker, facilmente, como todo projeto NodeJS, o que faz dele uma tecnologia bastante popular para a criação de microsserviços. Para começar a implementar nosso projeto de exemplo, crie um diretório vazio, abra com o VS Code e execute os comandos apresentados a seguir, de forma a inicializar o projeto e instalar a dependência do express.

Comando para instalação do Express

Atividade1

1. Crie um pasta chamada EXPRESS
2. **Digite no terminal do VSCODE**

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrador\Desktop\Pos Graducao\Express> npm init -y
```

npm init -y

npm install express

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

PS C:\Users\Administrador\Desktop\Pos Graducao\Express> npm init -y
```

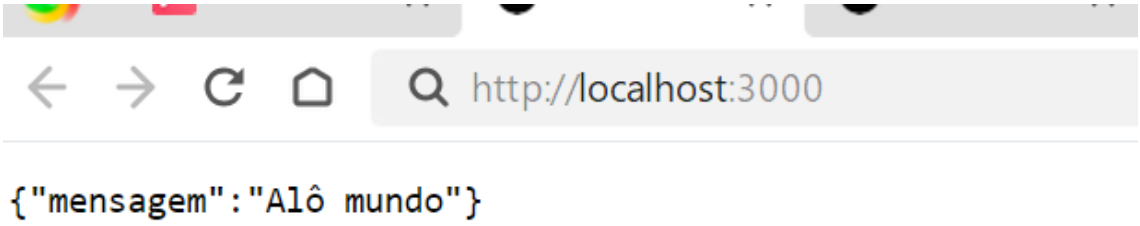
npm isntall

```
PS C:\Users\Administrador\Desktop\Pos Graducao\Express> npm install
```

3. Crie uma página chamada index1.js

```
// Importa o módulo express
const express = require('express');
const app = express();

// Define a porta que o servidor irá escutar
const port = 3000;

// Rota principal (GET /)
app.get('/', (req, res) => {
  // Envia uma resposta JSON com a mensagem
  res.json({ mensagem: "Alô mundo" }); // .json() já finaliza a resposta
});

// Inicia o servidor e escuta na porta especificada
app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

http://localhost:3000




```
PS C:\Users\Administrador\Desktop\Pos Graducao\Express> node index1
```

4. http://localhost:3000/

node i

Atividade2

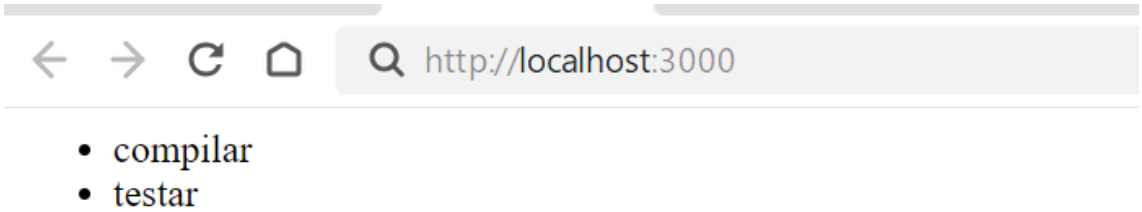
Tratamento de GET e POST

Na arquitetura REST, os métodos GET e POST são utilizados para consulta e criação de recursos, respectivamente. Para demonstrar o uso, vamos criar o arquivo index2.js, em nosso projeto, e utilizar o conteúdo da listagem seguinte.

```
// Importa o módulo Express (framework para aplicações web com Node.js)
const express = require('express');

// Cria uma instância do aplicativo Express
const app = express();

// Lista de tarefas simulada (poderia vir de um banco de dados no
futuro)
let tarefas = [
  { tarefa: "compilar" },
  { tarefa: "testar" }
];

// Rota GET para '/' e '/tarefas'
// Quando o usuário acessa essas rotas, a função abaixo é executada
app.get(['/', '/tarefas'], (req, res) => {

  // Início do conteúdo HTML a ser enviado
  let conteudo = "<html><body><ul>";

  // Para cada tarefa na lista, cria um item <li> dentro da lista <ul>
  for (let t of tarefas) {
    conteudo += `<li>${t.tarefa}</li>`;  // Exibe cada tarefa como item
de lista
  }

  // Fecha a lista e o HTML
  conteudo += "</ul></body></html>";

  // Envia a resposta com:
  // - status HTTP 200 (OK)
  // - tipo de conteúdo como HTML
  // - corpo da resposta sendo o HTML gerado acima
  res.status(200)
     .contentType('text/html')
```

```
    .send(conteudo);
});

// Inicia o servidor na porta 3000
// Exibe uma mensagem no console quando o servidor estiver ativo
app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

```
PS C:\Users\Administrador\Desktop\Pos Graducao\Express> node index2
```



- compilar
- testar

Atividade3

Os aplicativos web tradicionais são baseados em páginas, e suas interfaces cadastrais utilizam formulários HTML, trabalhando apenas com os métodos GET e POST. Isso é bem diferente do fornecimento de uma API, pois, aqui, existe o desenho da interface de usuário, exigindo que sejam construídos tanto o back-end quanto o front-end no mesmo sistema. Vamos iniciar a criação da terceira versão para nosso servidor, no arquivo index3.js, com o conteúdo da listagem seguinte.

Index3.js

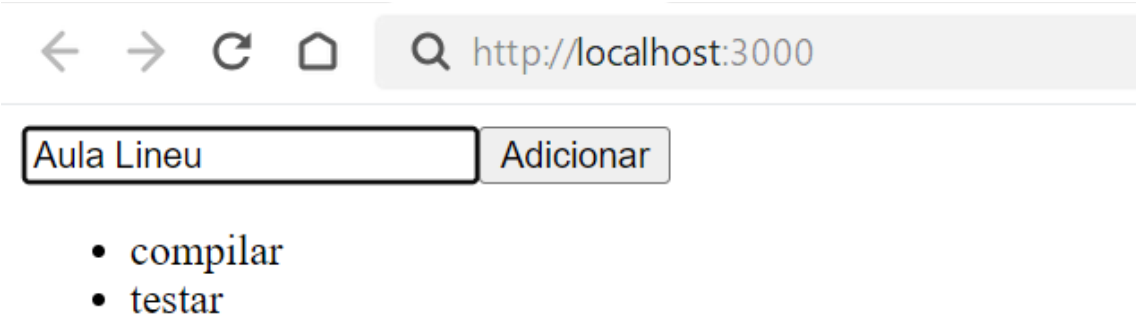
```
// Importa o módulo Express
const express = require('express');
const app = express();

// Middleware para interpretar dados enviados por formulários (POST)
app.use(express.urlencoded({ extended: true }));

// Lista inicial de tarefas
let tarefas = [
  { tarefa: "compilar" },
```

Servidor Express

```
  { tarefa: "testar" }
];

// Função que retorna a página HTML como string
const getPagina = () => {
  let conteudo = "<html><body>";

  // Formulário para adicionar nova tarefa
  conteudo += "<form method='post'>";
  conteudo += "<input type='text' name='tarefa' placeholder='Tarefa'>";
  conteudo += "<input type='submit' value='Adicionar'>";
  conteudo += "</form>";

  // Lista de tarefas
  conteudo += "<ul>";
  for (let t of tarefas) {
    conteudo += `<li>${t.tarefa}</li>`;
  }
  conteudo += "</ul>";

  conteudo += "</body></html>";
  return conteudo;
};

// Rota GET para exibir a página
app.get(['/', '/web/tarefas'], (req, res) => {
  res.status(200)
     .contentType('text/html')
     .send(getPagina());
});

// Rota POST para adicionar nova tarefa
app.post(['/', '/web/tarefas'], (req, res) => {
  // Adiciona a nova tarefa à lista, se não estiver vazia
  if (req.body.tarefa && req.body.tarefa.trim() !== "") {
    tarefas.push({ tarefa: req.body.tarefa.trim() });
  }

  // Reexibe a página com a nova tarefa incluída
  res.status(200)
     .contentType('text/html')
     .send(getPagina());
});

// Inicia o servidor na porta 3000
app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

A primeira grande mudança aqui é a do codificador, pois agora recebemos os dados a partir de um formulário, exigindo urlencoded. A captura dos valores enviados é similar à utilizada no REST, através do atributo body da requisição. Para os aplicativos web tradicionais, o retorno de uma requisição HTTP é uma página HTML, e, para diminuir o esforço de programação, criamos a função getPagina, que constrói a página de retorno, com o preenchimento de uma lista HTML com as tarefas. A página gerada também oferece um formulário com método POST, que, por não especificar a ação, chama a rota corrente. As respostas oferecidas via GET e POST são semelhantes, mas, ao tratar a chamada POST, que ocorre a partir do formulário, temos a inclusão da tarefa que foi enviada no repositório local. Note que temos duas rotas viáveis para cada um dos métodos, em que o conjunto de endpoints fica em um vetor.

```
Node.js v22.18.0
PS C:\Users\Administrador\Desktop\Pos Graducao\Express> node index3
Servidor rodando em http://localhost:3000
```



Atividade4

Utilizando Templates EJS

Ao trabalhar no modelo tradicional para web, fica fácil perceber que temos uma grande fragilidade na construção da resposta, pois, além de exigir extensos trechos de código, mesda a

sintaxe HTML com o JavaScript, dificultando muito a manutenção. Mais que isso, deixa a responsabilidade do design nas mãos do programador, enquanto poderia ser utilizado um profissional mais adequado na construção da interface, com ferramentas de maior produtividade. Uma solução elegante para esse problema é a adoção de templates HTML, que podem ser editados livremente pelos designers e preenchidos com os dados que são fornecidos pela camada de lógica, a qual é implementada em JavaScript no ambiente do NodeJS. Quem faz a substituição dos dados no template HTML, em tempo de execução, é um motor de visualização, e aqui iremos utilizar o Embedded JavaScript Templates (EJS). Vamos compreender seu funcionamento, de forma prática, começando com a instalação do EJS, em nosso projeto, através do comando seguinte.

Va no terminal do VSCODE

npm install ejs

Em seguida, crie o diretório **views**, no projeto; dentro dele, crie um arquivo template, com o nome **exemplo.ejs**, no qual será utilizado o código seguinte.

Temos uma página HTML, apesar da extensão ejs, com alguns trechos que terão substituição ao nível do servidor. São aceitas duas formas de interação: • Blocos executáveis, iniciados com ; • Blocos de substituição de valor, iniciados com .

Analisando a parte dinâmica do código, é executado um loop do tipo for, que engloba a linha da tabela através do uso de chaves, criando uma linha para cada iteração do loop. São percorridos todos os objetos de uma coleção de pessoas, fornecida por um controlador, e, para cada linha gerada, substitui-se o conteúdo interno das células pelos valores do nome e da idade da pessoa corrente.

Agora, vamos criar o arquivo **index4.js** e utilizar o seguinte conteúdo.

```
// Importa o módulo express
const express = require('express');
const app = express();

// Configura o mecanismo de visualização como EJS
app.set('view engine', 'ejs');

// Define o diretório onde os arquivos .ejs estão localizados
app.set('views', './views');

// Rota GET para /exemplo
app.get('/exemplo', (req, res) => {
```

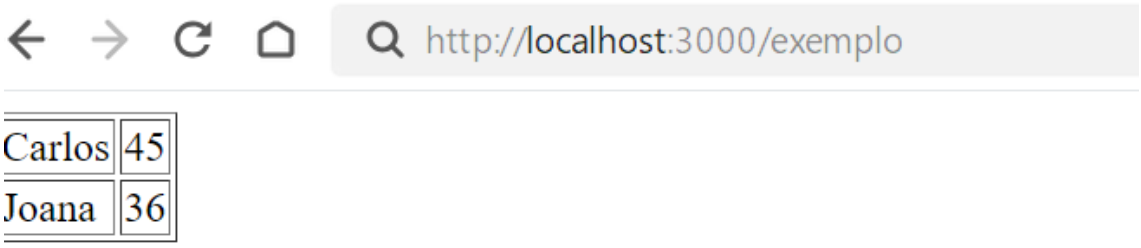
Servidor Express

```
  // Dados simulados para exibição na view
  const dados = [
    { nome: "Carlos", idade: 45 },
    { nome: "Joana", idade: 36 }
  ];

  // Renderiza o arquivo views/exemplo.ejs e envia os dados
  res.render('exemplo', { pessoas: dados });
});

// Inicia o servidor na porta 3000
app.listen(3000, () => {
  console.log('Servidor rodando em http://localhost:3000');
});
```

Acesse

```
PS C:\Users\Administrador\Desktop\Pos Graducao\Express> node index4
Servidor rodando em http://localhost:3000
```

http://localhost:3000/exemplo



No código, temos a configuração de view_engine como ejs e a definição do diretório de templates através do parâmetro views. Em seguida, implementamos o controlador que alimentará o template, respondendo no endereço /exemplo via método get. É muito importante observar como ocorre a ligação entre o controlador e o template, com a utilização do método render. Tudo que precisamos informar é o nome do template EJS e os dados que serão enviados para ele, no formato JSON, sendo fornecido, no exemplo, o vetor pessoas. Para executar o servidor, utilize o comando node index4, e o teste pode ser feito com o acesso ao endereço http://localhost:3000/exemplo, através de um navegador. Você deverá ter, como resultado, os dados das pessoas de exemplo em uma tabela HTML.

Atividade5

Aplicativo Web Tradicional

Vamos começar pelo controlador, no arquivo **index5.js**, utilizando o conteúdo da listagem seguinte.

Para um exemplo prático de utilização das tecnologias aqui descritas, vamos criar um aplicativo web tradicional, aproveitando o projeto corrente.

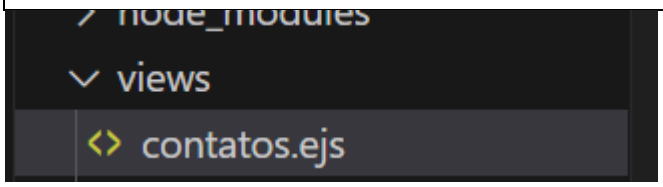
- O objetivo é apenas criar uma página para listagem, inclusão e exclusão de contatos.

Ao criar o arquivo index5.HTML

Insira o código

```
// Importa o módulo express
const express = require('express');
const app = express();

// Define a porta onde o servidor irá rodar
const port = 3000;

// Define o mecanismo de visualização como EJS
app.set('view engine', 'ejs');

// Define o diretório onde as views (arquivos .ejs) estão localizadas
app.set('views', './views');

// Middleware para interpretar dados de formulário enviados via POST
app.use(express.urlencoded({ extended: true }));

// Lista inicial de contatos
let contatos = [
  { nome: "XPTO", email: "xpto@xpto.com" }
];

// Rota GET para exibir os contatos
app.get('/contatos', (req, res) => {
  // Se houver parâmetro delId na URL, remove o contato correspondente
  if (req.query.delId) {
    contatos.splice(req.query.delId, 1);
```

```
  }

  // Renderiza a view 'contatos.ejs' passando a lista de contatos
  res.render('contatos', { contatos: contatos });
});

// Rota POST para adicionar um novo contato
app.post('/contatos', (req, res) => {
  // Adiciona um novo contato com os dados enviados pelo formulário
  contatos.push({
    nome: req.body.n1,
    email: req.body.e1
  });

  // Renderiza a view novamente com a lista atualizada
  res.render('contatos', { contatos: contatos });
});

// Inicia o servidor e escuta na porta definida
app.listen(port, () => {
  console.log(`Servidor rodando em http://localhost:${port}`);
});
```



Views/contatos.ejs

```
<!DOCTYPE html>
<html>
<head>
  <meta charset="UTF-8">
  <title>Contatos</title>
</head>
<body>
  <h1>Lista de Contatos</h1>

  <form method="POST" action="/contatos">
    <input type="text" name="n1" placeholder="Nome" required>
    <input type="email" name="e1" placeholder="Email" required>
    <button type="submit">Adicionar</button>
  </form>
```

```
  <ul>
    <% contatos.forEach((c, index) => { %>
      <li>
        <strong><%= c.nome %></strong> - <%= c.email %>
        <a href="/contatos?delId=<%= index %>">[Remover]</a>
      </li>
    <% }) %>
  </ul>
</body>
</html>
```

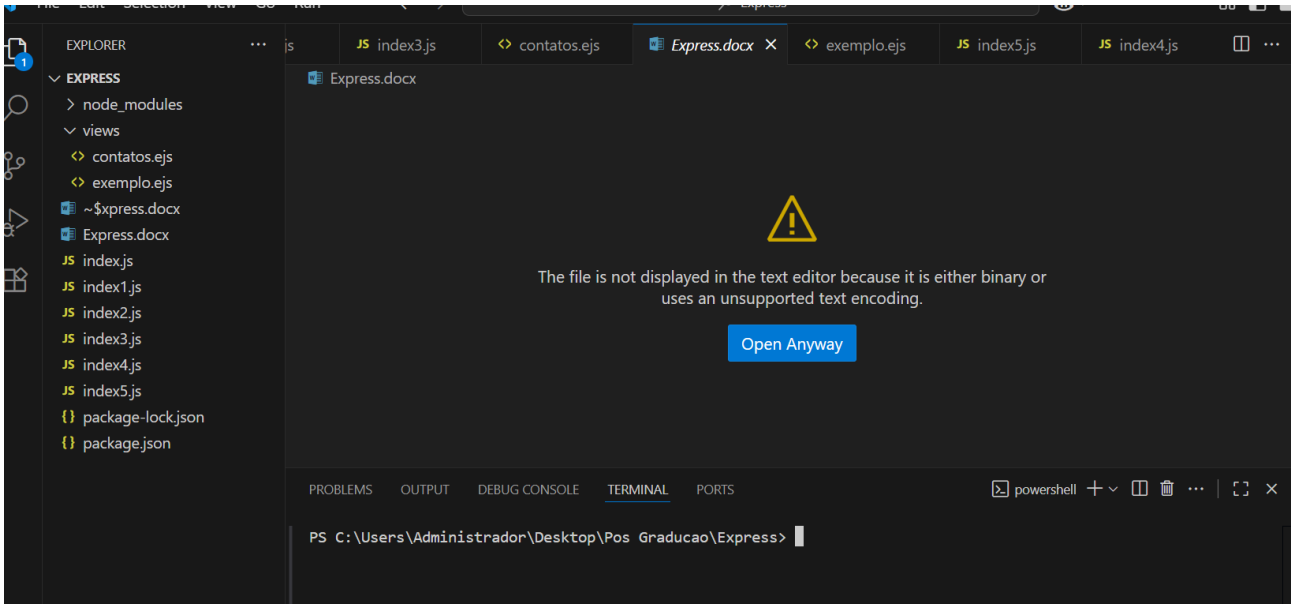
Acesse http://localhost:3000/contatos

Considerações Finais da Aula

Os servidores minimalistas são uma estratégia de desenvolvimento cada vez mais comum, estando presente em plataformas como: Python, com o Flask; Java, em que temos o Spring Boot; e NodeJS, que oferece o framework Express. São produtos similares, oferecendo o tratamento para requisições HTTP para todos os métodos do protocolo, além de permitirem, em maior ou menor escala, o uso de funcionalidades mais avançadas, como codificadores e autenticação OAuth. Para um sistema web tradicional, baseado em páginas, o requisito mínimo é o tratamento de rotas via métodos GET e POST, algo que o Express oferece, de forma simples, robusta e flexível, através de seu modelo de roteamento. Para facilitar a criação desse tipo de

sistema, o uso de templates baseados em HTML é um grande facilitador, além de separar a lógica e o design de telas, permitindo especializar o desenvolvimento por perfil de profissional. Com os conhecimentos adquiridos, fomos capazes de criar um aplicativo web simples, mas que incorpora as técnicas necessárias para a definição de sistemas maiores, nos quais teríamos diversas páginas, mas continuaríamos a utilizar apenas os métodos GET e POST do protocolo HTTP.

Como deve ficar o arquivo



Como iniciar o terminal

Aperta o crtrl+”

Servidor Express

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS    po

PS C:\Users\Administrador\Desktop\Pos Graducao\Express>
```